

Movie Recommendation System

System Architecture & Autonomous Antigravity Execution Goal Prompt

Document Version: 1.0.0	Engine: Google Antigravity	Artifact: Workpack Specification
Domain: Recommender Systems	Dataset: MovieLens 100K / 1M	Algorithms: Hybrid Filtering (CF, Content TF-IDF, Neural SVD)

1. PROJECT OVERVIEW & ENVIRONMENT SPECIFICATIONS

The **Movie Recommendation System** project builds an end-to-end, production-ready hybrid recommendation engine using the MovieLens dataset. It integrates Content-Based Filtering (Cosine Similarity over TF-IDF metadata) and Collaborative Filtering (Matrix Factorization / SVD via Surprise or PyTorch) into a unified hybrid scoring pipeline.

Component	Technology / Paradigm	Description
Content-Based Engine	TF-IDF + Cosine Similarity	Computes text similarity vectors based on movie genres, overview, keywords, and cast.
Collaborative Engine	Matrix Factorization (SVD)	Decomposes User-Item interaction matrix to learn latent factors and predict rating preferences.
Hybrid Engine	Weighted Blending / Stacking	Combines item metadata similarities with user interaction predictions to eliminate cold-start issues.
Evaluation Metrics	RMSE, MAE, Precision@K, Recall@K	Evaluates rating accuracy alongside ranking performance for top-K recommendations.

2. ANTIGRAVITY GOAL EXECUTION PROMPT

Copy and paste the exact block below into Google Antigravity in `/goal` mode to autonomously construct the repository:

```
/goal Build a complete, production-ready Movie Recommendation System repository in Python using the MovieLens dataset, featuring Hybrid Recommendation architecture (Content-Based Filtering via TF-IDF/Cosine Similarity + Collaborative Filtering via SVD/Matrix Factorization), evaluation benchmarks (RMSE, MAE, Precision@K, Recall@K), a FastAPI backend, an interactive Streamlit Netflix-style UI, unit tests, Docker setup, and a GitHub-ready README.md.

### DIRECTORY STRUCTURE TO CREATE:
movie_recommender_project/
├── data/
│   ├── raw/                # Downloaded MovieLens dataset files
│   └── processed/           # Processed sparse matrices & item metadata dataframes
├── models/                  # Saved svd_model.pkl, tfidf_vectorizer.pkl,
    similarity_matrix.npz
├── src/
│   ├── __init__.py
│   ├── data_loader.py      # Fetch MovieLens 100K/1M dataset automatically or fallback synth
    generator
│   ├── preprocessing.py    # Text cleaning, genre encoding, and sparse interaction matrix
    creation
│   ├── content_engine.py   # TF-IDF vectorizer and Cosine Similarity calculation for item
    features
│   └── collaborative_engine.py # SVD Matrix Factorization / PyTorch Embedding training
```

```

|   |— hybrid_engine.py      # Weighted ensemble scoring combining content + collaborative
predictions
|   |— evaluate.py          # Compute RMSE, MAE, MAP@K, Precision@K, Recall@K & plot top-K
curves
|— app/
|   |— api.py              # FastAPI server (POST /recommend, POST /predict_rating, GET /
health)
|   |— ui.py              # Netflix-style Streamlit web dashboard with poster previews &
dynamic search
|— tests/
|   |— test_recommender.py
|   |— test_api.py
|— requirements.txt
|— Dockerfile
|— main.py                # CLI pipeline runner (--mode train|evaluate|serve)
|— README.md              # Comprehensive, GitHub-ready README file

```

SPECIFIC REQUIREMENTS FOR GENERATED README.md:

1. **Badges:** Python 3.10+, Scikit-Learn, PyTorch/Surprise, FastAPI, License MIT.
2. **Project Overview:** Hybrid recommendation strategy addressing cold-start and data sparsity.
3. **Architecture Diagram:** Mermaid.js flowchart mapping User Input -> Dual Pipelines (Content TF-IDF + Collaborative SVD) -> Hybrid Blender -> Top-K Ranking -> Streamlit/API UI.
4. **Metrics Table:** Comparative benchmarking table showing Content-Only vs SVD Collaborative vs Hybrid ensemble performance (Target RMSE < 0.90, Precision@10 > 0.75).
5. **Quickstart Instructions:** Environment setup, data fetching commands, training CLI, API endpoint specs, Streamlit dashboard deployment, and Docker execution.
6. **API Specifications:** Request JSON schema (`user\_id`, `movie\_title`, `top\_n`, `alpha\_weight`) and response payload with recommended movies and predicted scores.

WORKFLOW STEPS:

1. Implement `src/data\_loader.py` and `src/preprocessing.py` handling data cleanups and matrices.
2. Implement `src/content\_engine.py` using TF-IDF feature extraction.
3. Implement `src/collaborative\_engine.py` using SVD matrix factorization.
4. Implement `src/hybrid\_engine.py` blending content and interaction scores seamlessly.
5. Build evaluation suite in `src/evaluate.py` exporting metrics and validation plots.
6. Build FastAPI backend (`app/api.py`) and Streamlit web interface (`app/ui.py`).
7. Construct Dockerfile exposing ports 8000 and 8501.
8. Implement unit tests in `tests/` and construct the complete `README.md`.
9. Execute `pytest` to confirm end-to-end repository build passing status.